



Instituto Politécnico Nacional
"La Técnica al Servicio de la Patria"



robótica y mecatrónica



INSTITUTO POLITÉCNICO NACIONAL CENTRO DE INVESTIGACIÓN EN COMPUTACIÓN LAB. ROBÓTICA Y MECATRÓNICA

PRACTICAS

DRON

Título específico de lo que ud. hizo

Autores:

Rodrigo Emmanuel Arellano Flores

Profesores:

Prof. Dr. Juan Humberto Sossa Azuela

CDMX.
12 de diciembre de 2025

Índice

1. Resumen.	3
2. Introducción.	3
3. Información preliminar.	3
3.1. Componentes del cuadricóptero	3
3.2. Control del cuadricóptero	4
3.3. Procesamiento de sensores	4
3.3.1. IMU	4
3.3.2. Barómetro	5
4. Metodologías.	5
4.1. Dinámica	5
4.1.1. Linealización	6
4.2. Algoritmos de control	6
4.2.1. PD	6
4.2.2. PID	7
4.2.3. LQI	7
4.3. Resultados.	8
5. Conclusiones.	8
Apéndices	10
A. Memoria de cálculos.	10
B. Algoritmos.	10
C. Planos	16

1. Resumen.

Lo que se hizo, para qué sirve, cómo se hizo, qué resultados se obtuvieron, y porqué es importante haber logrado esto. (de 120 palabras máximo). Ejemplo de cómo dar una cita bibliográfica

2. Introducción.

Lo que Ud. escriba aquí debe responder a las preguntas hechas por cualquier lector en el mundo:

- ¿De qué se trata este trabajo?
- ¿Por qué estas personas hicieron este trabajo?
- ¿Por qué vale la pena leerlo, y a quienes le puede interesar leerlo?
- ¿Qué tiene de original o importante lo hecho aquí?
- ¿Qué tan difícil fue lograr los resultados que se muestran aquí?
- Puede incluir dibujos si los necesita para explicar y/o fotografías

3. Información preliminar.

No es el "Marco Teórico" que normalmente se llena con copiar y pegar de internet en trabajos de otros cursos. Por eso tiene otro nombre.

- ¿Qué otros trabajos o desarrollos similares se han hecho?
- ¿Cuál es la diferencia de este respecto a aquellos?
- ¿Qué modelos, teorías o cuerpos de conocimiento hay que tener presentes para leer el resto del trabajo presentado.
- NO REPITA lo que otros ya escribieron y presentaron en documentos digitales (web/o textos).
- Describa lo que se necesita para entender SU trabajo en sus PROPIAS palabras.
- Necesariamente debe nombrar los documentos o autores a los que hace referencia (citelos) y sobretodo genere sus propios análisis al respecto.
- Solo haga mención explícita de lo que Ud. VA A usar en su trabajo.
- Temas relacionados pero QUE NO se van a usar pueden mencionarse como citas.

3.1. Componentes del cuadricóptero

El dron cuenta con los siguientes componentes:

- 4 Motores sin escobillas (BLDC)
- 4 Control de velocidad electrónico (ESC)
- 1 batería LiPo
- 1 Computadora de vuelo, cuenta con:
 - IMU (acelerometro, giroscopio y magnetómetro)
 - barómetro
- 1 Chasis
- 2 Helices CW
- 2 Helices CCW

3.2. Control del cuadricóptero

Se puede linealizar el modelo del dron mediante la serie de Taylor [1] mediante

$$\dot{\mathbf{x}} = \left. \frac{\partial}{\partial \mathbf{x}} f(\mathbf{x}, \mathbf{u}) \right|_{(\mathbf{x}_0, \mathbf{u}_0)} (\mathbf{x} - \mathbf{x}_0) + \left. \frac{\partial}{\partial \mathbf{u}} f(\mathbf{x}, \mathbf{u}) \right|_{(\mathbf{x}_0, \mathbf{u}_0)} (\mathbf{u} - \mathbf{u}_0) = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u} \quad (1)$$

$$\mathbf{x}_0 = \mathbf{0}_{12 \times 1}$$

$$\mathbf{u}_0 = [mg \ 0 \ 0 \ 0]^T$$

Se puede convertir el espacio de estados a funciones de transferencia mediante

$$\mathbf{G}(s) = \mathbf{C} [s\mathbf{I} - \mathbf{A}]^{-1} \mathbf{B} + \mathbf{D} \quad (2)$$

El controlador PD tiene la siguiente estructura

$$PD(s) = K_P + K_D s = K_D (s + a) \quad (3)$$

$$K_P = aK_D$$

El controlador PID tiene la siguiente estructura

$$PID(s) = K_P + \frac{K_I}{s} + K_D s = K_D \frac{(s + a)(s + b)}{s} \quad (4)$$

$$K_I = abK_D$$

$$K_P = (a + b)K_D$$

Modelo Interno para entrada escalón [2]

$$\begin{bmatrix} \dot{\mathbf{e}}(t) \\ \dot{\mathbf{z}}(t) \end{bmatrix} = \begin{bmatrix} 0 & \mathbf{C} \\ 0 & \mathbf{A} \end{bmatrix} \begin{bmatrix} \mathbf{e}(t) \\ \mathbf{z}(t) \end{bmatrix} + \begin{bmatrix} 0 \\ \mathbf{B} \end{bmatrix} \mathcal{W}(t) \quad (5)$$

$$\mathbf{e}(t) = y(t) - r(t)$$

$$\mathbf{z}(t) = \dot{\mathbf{x}}(t)$$

$$\mathcal{W}(t) = -\mathbf{K}_1 \mathbf{e}(t) - \mathbf{K}_2 \mathbf{z}(t)$$

3.3. Procesamiento de sensores

3.3.1. IMU

Se pueden estimar los ángulos de orientación roll y pitch mediante un filtro complementario (6) para fusionar la información de giroscopio y acelerómetros, sin embargo aunque es sencillo de implementar y de muy bajo costo computacional tiende a variar con rápidos movimientos.

$$\theta = \alpha(\theta(k-1) + \omega_{gyro}T) + (1 - \alpha)\theta_{acc}(k) \quad (6)$$

$$\phi_{acc} = \frac{acc_y}{\sqrt{acc_x^2 + acc_z^2}} \quad (7)$$

$$\theta_{acc} = -\frac{acc_x}{\sqrt{acc_y^2 + acc_z^2}} \quad (8)$$

Una mejor forma de obtener la orientación es mediante filtro Madgwick, este proporciona roll, pitch y yaw [3], este se basa en un algoritmo de gradiente descendente. Aunque es mas costoso es más preciso con un rendimiento comparable a un filtro Kalman extendido [4]. El algoritmo proporciona la orientación en cuaterniones, los cuales pueden ser convertidos a ángulos de Euler mediante.

$$\phi = \text{atan2}(2(q_0q_1 + q_2q_3), 1 - 2(q_1^2 + q_2^2)) \quad (9)$$

$$\theta = \text{asin}(2(q_0q_2 - q_3q_1)) \quad (10)$$

$$\psi = \text{atan2}(2(q_0q_3 + q_1q_2), 1 - 2(q_2^2 + q_3^2)) \quad (11)$$

3.3.2. Barómetro

La altura ortométrica [5] se calcula con la información del barómetro mediante la siguiente ecuación

$$H_b = \frac{T_s}{k_T} \left(\frac{p_b}{p_s}^{\frac{Rk_T}{g_0}} - 1 \right) \quad (12)$$

Donde:

R 287.1 J/kgK

k_T 6.5×10^{-3} K/m

g_0 9.80665 m/s²

p_s 101.325 kPa

T_s 288.15 K

4. Metodologías.

- Diga cómo se hizo lo que ud. (o uds) hicieron.
- Sea claro y preciso.
- Si alguien quiere repetir su trabajo, debe poder usar esto como guía.
- Registre cada una de las iteraciones seguidas por sus procesos de diseño.
- Si hubo trabajo en equipo, aquí debe quedar especificada la participación de cada uno.
- Aquí deben estar registradas (como mínimo) los siguientes :
 1. Diagramas de cuerpo libre,
 2. diagramas de cortante y momentos,
 3. diagramas de torsión,
 4. cálculos de velocidades,
 5. análisis de aceleraciones y
 6. análisis de fuerzas,
 7. análisis de fatiga si es el caso,
 8. calculo de los factores de Marin y
 9. uso de variables estocásticas,
 10. análisis de pandeo,
 11. diseño de ejes,
 12. diseño de sujetadores y uniones si los hay, etc

4.1. Dinámica

Relación entre entradas de control y actuadores

$$\begin{bmatrix} F \\ \tau_x \\ \tau_y \\ \tau_z \end{bmatrix} = \begin{bmatrix} K_F & K_F & K_F & K_F \\ -K_F L & -K_F L & K_F L & K_F L \\ -K_F L & K_F L & K_F L & -K_F L \\ -K_M & K_M & -K_M & K_M \end{bmatrix} \begin{bmatrix} \omega_1^2 \\ \omega_2^2 \\ \omega_3^2 \\ \omega_4^2 \end{bmatrix} \quad (13)$$

$$\therefore \begin{bmatrix} \omega_1^2 \\ \omega_2^2 \\ \omega_3^2 \\ \omega_4^2 \end{bmatrix} = \begin{bmatrix} \frac{1}{4K_F} & -\frac{1}{4K_F L} & -\frac{1}{4K_F L} & -\frac{1}{4K_M} \\ \frac{1}{4K_F} & -\frac{1}{4K_F L} & \frac{1}{4K_F L} & \frac{1}{4K_M} \\ \frac{1}{4K_F} & \frac{1}{4K_F L} & \frac{1}{4K_F L} & -\frac{1}{4K_M} \\ \frac{1}{4K_F} & \frac{1}{4K_F L} & -\frac{1}{4K_F L} & \frac{1}{4K_M} \end{bmatrix} \begin{bmatrix} F \\ \tau_x \\ \tau_y \\ \tau_z \end{bmatrix} \quad (14)$$

Cuadro 1: Parámetros del dron

m	1.6	kg
I_{xx}	0.021915027	kgm ²
I_{yy}	0.022124548	kgm ²
I_{zz}	0.042349865	kgm ²
L	0.23	m
K_F	6.11×10^{-8}	N/ms

4.1.1. Linealización

Mediante la serie de Taylor (1) el modelo del cuadricóptero se puede expresar de forma lineal de la siguiente forma

$$\begin{bmatrix} \dot{\mathbf{r}} \\ \ddot{\mathbf{r}} \\ \dot{\boldsymbol{\eta}} \\ \ddot{\boldsymbol{\eta}} \end{bmatrix} = \begin{bmatrix} \mathbf{0}_{3 \times 3} & \mathbf{I}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} \\ \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & G(\psi_d) & \mathbf{0}_{3 \times 3} \\ \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{I}_{3 \times 3} \\ \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} \end{bmatrix} \begin{bmatrix} \mathbf{r} \\ \dot{\mathbf{r}} \\ \boldsymbol{\eta} \\ \dot{\boldsymbol{\eta}} \end{bmatrix} + \begin{bmatrix} \mathbf{0}_{5 \times 1} & \mathbf{0}_{5 \times 3} \\ m^{-1} & \mathbf{0}_{1 \times 3} \\ \mathbf{0}_{3 \times 1} & \mathbf{0}_{3 \times 3} \\ \mathbf{0}_{3 \times 1} & \mathbf{I}_d \end{bmatrix} \begin{bmatrix} F \\ \boldsymbol{\tau} \end{bmatrix} \quad (15)$$

donde:

$$\mathbf{r} = \begin{bmatrix} x \\ y \\ z \end{bmatrix}, \quad \boldsymbol{\eta} = \begin{bmatrix} \phi \\ \theta \\ \psi \end{bmatrix}, \quad \boldsymbol{\tau} = \begin{bmatrix} \tau_x \\ \tau_y \\ \tau_z \end{bmatrix}$$

$$G(\psi_d) = \begin{bmatrix} 0 & g & 0 \\ -g & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \quad \mathbf{I}_d = \text{diag} \left(I_{xx}^{-1}, I_{yy}^{-1}, I_{zz}^{-1} \right)$$

4.2. Algoritmos de control

4.2.1. PD

El controlador se ajusta mediante las funciones de transferencia, que se obtienen mediante (2), con

$$\mathbf{C} = \begin{bmatrix} \mathbf{I}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} \\ \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \mathbf{I}_{3 \times 3} & \mathbf{0}_{3 \times 3} \end{bmatrix}$$

$$\mathbf{D} = \mathbf{0}_{6 \times 4}$$

$$G_x(s) = \frac{g}{I_{yy}s^4} \quad (16)$$

$$G_y(s) = -\frac{g}{I_{xx}s^4} \quad (17)$$

$$G_z(s) = \frac{1}{ms^2} \quad (18)$$

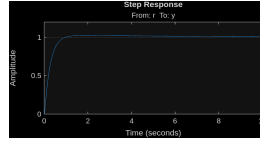
$$G_\phi(s) = \frac{1}{I_{xx}s^2} \quad (19)$$

$$G_\theta(s) = \frac{1}{I_{yy}s^2} \quad (20)$$

$$G_\psi(s) = \frac{1}{I_{zz}s^2} \quad (21)$$

Dado que no se puede medir los desplazamientos en x y y , solo se realizan 4 de los 6 controladores (control de orientación y altura). Dado que el cambio de la velocidad de los motores no es instantáneo, las ecuaciones (21)-(21) se multiplican por (22), que es la función de transferencia de los motores, para incluir este efecto.

$$\frac{K_m}{s + K_m} \quad (22)$$



(a) Altura

Figura 1: Respuesta escalón PD

Para ajustar los controladores se utiliza el método de colocación de las raíces, de modo que (18) a (21) se convierten en

$$K_{Dz} \frac{(s + a_z) \frac{K_m}{m}}{s^2(s + K_m)} \quad (23)$$

$$K_{D\phi} \frac{(s + a_\phi) \frac{K_m}{I_{xx}}}{s^2(s + K_m)} \quad (24)$$

$$K_{D\theta} \frac{(s + a_\theta) \frac{K_m}{I_{yy}}}{s^2(s + K_m)} \quad (25)$$

$$K_{D\psi} \frac{(s + a_\psi) \frac{K_m}{I_{zz}}}{s^2(s + K_m)} \quad (26)$$

Con la ayuda MATLAB se ajustan los ceros y ganancias hasta obtener los siguiente resultados

4.2.2. PID

De forma similar se ajustan las ganancias del controlador, para este caso se usan

$$K_{Dz} \frac{(s + a_z)(s + b_z) \frac{K_m}{m}}{s^2(s + K_m)} \quad (27)$$

$$K_{D\phi} \frac{(s + a_\phi)(s + b_\phi) \frac{K_m}{I_{xx}}}{s^2(s + K_m)} \quad (28)$$

$$K_{D\theta} \frac{(s + a_\theta)(s + b_\theta) \frac{K_m}{I_{yy}}}{s^2(s + K_m)} \quad (29)$$

$$K_{D\psi} \frac{(s + a_\psi)(s + b_\psi) \frac{K_m}{I_{zz}}}{s^2(s + K_m)} \quad (30)$$

4.2.3. LQI

Para realizar seguimiento de una referencia se diseña un controlador de modelo interno (5) y se calculan las ganancias como LQR, este al contener un integrador se convierte en LQI, se calculan las ganancias para los siguientes valores

$$\mathbf{AA} = \begin{bmatrix} 0 & 0 & 0 & 0 & | & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & | & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & | & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & | & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ -\frac{0}{0} & -\frac{0}{0} & -\frac{0}{0} & -\frac{0}{0} & | & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & | & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & | & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & | & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & | & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & | & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & | & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & | & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$\mathbf{B}\mathbf{B} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ \hline 0 & 0 & 0 & 0 \\ 0,625 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0,021915027 & 0 & 0 \\ 0 & 0 & 0,022124548 & 0 \\ 0 & 0 & 0 & 0,042349865 \end{bmatrix}$$

$$\mathbf{Q} = \text{diag}(8, 50, 500, 500, 0,5, 0,5, 200, 200, 200, 10, 10, 10)$$

$$\mathbf{R} = \text{diag}(0,0001, 0,0001, 0,0001, 0,0001)$$

Las ganancias del controlador son:

$\mathbf{K}$

4.3. Resultados.

Describa los resultados obtenidos. Valores de Fuerzas, Cargas, Valores de esfuerzos hallados, si es el caso las dimensiones calculadas, espesores, longitudes, y planos de diseño resultantes.

5. Conclusiones.

Explique por qué su diseño sirve (o nó) y qué hace falta por hacer. Responda las expectativas generadas en la Introducción. Analice sus resultados críticamente y diga si son creíbles y responda qué tan seguras son sus propuestas.

Referencias

- [1] C. Powers, D. Mellinger y V. Kumar, «Quadrotor Kinematics and Dynamics,» en *Handbook of Unmanned Aerial Vehicles*, Springer, Dordrecht, 2015, págs. 307-328, ISBN: 978-90-481-9707-1. DOI: [10.1007/978-90-481-9707-1_71](https://doi.org/10.1007/978-90-481-9707-1_71). visitado 7 de dic. de 2025. dirección: https://link.springer.com/rwe/10.1007/978-90-481-9707-1_71.
- [2] R. C. Dorf y R. H. Bishop, «Internal Model Design,» en *Modern Control Systems*, 14.^a ed., Pearson, 2022, págs. 845-848, ISBN: 978-1-292-42235-0.
- [3] S. O. H. Madgwick, A. J. L. Harrison y R. Vaidyanathan, «Estimation of IMU and MARG Orientation Using a Gradient Descent Algorithm,» en *2011 IEEE International Conference on Rehabilitation Robotics*, jun. de 2011, págs. 1-7. DOI: [10.1109/ICORR.2011.5975346](https://doi.org/10.1109/ICORR.2011.5975346). visitado 7 de dic. de 2025. dirección: <https://ieeexplore.ieee.org/document/5975346>.
- [4] A. Cavallo et al., «Experimental Comparison of Sensor Fusion Algorithms for Attitude Estimation,» *IFAC Proceedings Volumes*, 19th IFAC World Congress, vol. 47, n.º 3, págs. 7585-7591, 1 de ene. de 2014, ISSN: 1474-6670. DOI: [10.3182/20140824-6-ZA-1003.01173](https://doi.org/10.3182/20140824-6-ZA-1003.01173). visitado 8 de dic. de 2025. dirección: <https://www.sciencedirect.com/science/article/pii/S1474667016428089>.
- [5] P. D. Groves, «Barometric Altimeter,» en *Principles of GNSS, inertial, and multisensor integrated systems*, 2.^a ed., Artech House, 2013, págs. 230-231, ISBN: 978-1-60807-005-3.

When esto es lo que pasa cuando nunca te rindes:



Figura 2: Esto es lo que pasa cuando nunca te rindes

Apéndices

A. Memoria de cálculos.

Digitalice sus manuscritos y los procedimientos de cálculo utilizados. Deben ser legibles y claros.
 ¡Documentos escaneados! con algunas explicaciones.
 Si prefiere escribir todos sus calculos usando fórmulas de LATEX hágalo a través de la metodología.

B. Algoritmos.

Listing 1: AHRS Madwgick.h

```

1  //
   =====

2  // MadgwickAHRS.h
3  //
   =====

4  //
5  // Implementation of Madgwick's IMU and AHRS algorithms.
6  // See: http://www.x-io.co.uk/node/8#open\_source\_ahrs\_and\_imu\_algorithms
7  //
8  // Date          Author          Notes
9  // 29/09/2011    SOH Madgwick     Initial release
10 // 02/10/2011    SOH Madgwick     Optimised for reduced CPU load
11 //
12 //
   =====

13 #ifndef MadgwickAHRS_h
14 #define MadgwickAHRS_h
15
16 //
   -----

17 // Variable declaration
18
19 extern volatile float beta;           // algorithm gain
20 extern volatile float q0, q1, q2, q3; // quaternion of sensor frame
    relative to auxiliary frame
21
22 //
   -----

23 // Function declarations
24
25 void MadgwickAHRSupdate(float gx, float gy, float gz, float ax, float ay,
    float az, float mx, float my, float mz);
26 void MadgwickAHRSupdateIMU(float gx, float gy, float gz, float ax, float ay,
    float az);
27
28 #endif

```

29 `//`

`=====`

30 `// End of file`

31 `//`

`=====`

Listing 2: AHRS Madwgick.c

1 `//`

`=====`

2 `// MadgwickAHRS.c`

3 `//`

`=====`

4 `//`

5 `// Implementation of Madgwick's IMU and AHRS algorithms.`

6 `// See: http://www.x-io.co.uk/node/8#open\_source\_ahrs\_and\_imu\_algorithms`

7 `//`

8 `// Date Author Notes`

9 `// 29/09/2011 SOH Madgwick Initial release`

10 `// 02/10/2011 SOH Madgwick Optimised for reduced CPU load`

11 `// 19/02/2012 SOH Madgwick Magnetometer measurement is normalised`

12 `//`

13 `//`

`=====`

14 `//`

15 `//`

`-----`

16 `// Header files`

17

18 `#include "AHRS/MadgwickAHRS.h"`

19 `#include <math.h>`

20

21 `//`

`-----`

22 `// Definitions`

23

24 `#define sampleFreq 512.0f // sample frequency in Hz`

25 `#define betaDef 0.1f // 2 * proportional gain`

26

27 `//`

`-----`

28 `// Variable definitions`

29

30 `volatile float beta = betaDef; // 2 *`

`proportional gain (Kp)`

31 `volatile float q0 = 1.0f, q1 = 0.0f, q2 = 0.0f, q3 = 0.0f; // quaternion of
sensor frame relative to auxiliary frame`

32

```

33  //
34  // Function declarations
35
36  float invSqrt(float x);
37
38  //
=====
39  // Functions
40
41  //
-----

42  // AHRS algorithm update
43
44  void MadgwickAHRSupdate(float gx, float gy, float gz, float ax, float ay,
45                          float az, float mx, float my, float mz) {
46      float recipNorm;
47      float s0, s1, s2, s3;
48      float qDot1, qDot2, qDot3, qDot4;
49      float hx, hy;
50      float _2q0mx, _2q0my, _2q0mz, _2q1mx, _2bx, _2bz, _4bx, _4bz, _2q0, _2q1
51          , _2q2, _2q3, _2q0q2, _2q2q3, q0q0, q0q1, q0q2, q0q3, q1q1, q1q2,
52          q1q3, q2q2, q2q3, q3q3;
53
54      // Use IMU algorithm if magnetometer measurement invalid (avoids NaN in
55      // magnetometer normalisation)
56      if((mx == 0.0f) && (my == 0.0f) && (mz == 0.0f)) {
57          MadgwickAHRSupdateIMU(gx, gy, gz, ax, ay, az);
58          return;
59      }
60
61      // Rate of change of quaternion from gyroscope
62      qDot1 = 0.5f * (-q1 * gx - q2 * gy - q3 * gz);
63      qDot2 = 0.5f * (q0 * gx + q2 * gz - q3 * gy);
64      qDot3 = 0.5f * (q0 * gy - q1 * gz + q3 * gx);
65      qDot4 = 0.5f * (q0 * gz + q1 * gy - q2 * gx);
66
67      // Compute feedback only if accelerometer measurement valid (avoids NaN
68      // in accelerometer normalisation)
69      if(!((ax == 0.0f) && (ay == 0.0f) && (az == 0.0f))) {
70
71          // Normalise accelerometer measurement
72          recipNorm = invSqrt(ax * ax + ay * ay + az * az);
73          ax *= recipNorm;
74          ay *= recipNorm;
75          az *= recipNorm;
76
77          // Normalise magnetometer measurement
78          recipNorm = invSqrt(mx * mx + my * my + mz * mz);
79          mx *= recipNorm;
80          my *= recipNorm;
81          mz *= recipNorm;

```

```

77
78 // Auxiliary variables to avoid repeated arithmetic
79 _2q0mx = 2.0f * q0 * mx;
80 _2q0my = 2.0f * q0 * my;
81 _2q0mz = 2.0f * q0 * mz;
82 _2q1mx = 2.0f * q1 * mx;
83 _2q0 = 2.0f * q0;
84 _2q1 = 2.0f * q1;
85 _2q2 = 2.0f * q2;
86 _2q3 = 2.0f * q3;
87 _2q0q2 = 2.0f * q0 * q2;
88 _2q2q3 = 2.0f * q2 * q3;
89 q0q0 = q0 * q0;
90 q0q1 = q0 * q1;
91 q0q2 = q0 * q2;
92 q0q3 = q0 * q3;
93 q1q1 = q1 * q1;
94 q1q2 = q1 * q2;
95 q1q3 = q1 * q3;
96 q2q2 = q2 * q2;
97 q2q3 = q2 * q3;
98 q3q3 = q3 * q3;
99
100 // Reference direction of Earth's magnetic field
101 hx = mx * q0q0 - _2q0my * q3 + _2q0mz * q2 + mx * q1q1 + _2q1 * my *
    q2 + _2q1 * mz * q3 - mx * q2q2 - mx * q3q3;
102 hy = _2q0mx * q3 + my * q0q0 - _2q0mz * q1 + _2q1mx * q2 - my * q1q1
    + my * q2q2 + _2q2 * mz * q3 - my * q3q3;
103 _2bx = sqrt(hx * hx + hy * hy);
104 _2bz = -_2q0mx * q2 + _2q0my * q1 + mz * q0q0 + _2q1mx * q3 - mz *
    q1q1 + _2q2 * my * q3 - mz * q2q2 + mz * q3q3;
105 _4bx = 2.0f * _2bx;
106 _4bz = 2.0f * _2bz;
107
108 // Gradient decent algorithm corrective step
109 s0 = -_2q2 * (2.0f * q1q3 - _2q0q2 - ax) + _2q1 * (2.0f * q0q1 +
    _2q2q3 - ay) - _2bz * q2 * (_2bx * (0.5f - q2q2 - q3q3) + _2bz *
    (q1q3 - q0q2) - mx) + (-_2bx * q3 + _2bz * q1) * (_2bx * (q1q2 -
    q0q3) + _2bz * (q0q1 + q2q3) - my) + _2bx * q2 * (_2bx * (q0q2 +
    q1q3) + _2bz * (0.5f - q1q1 - q2q2) - mz);
110 s1 = _2q3 * (2.0f * q1q3 - _2q0q2 - ax) + _2q0 * (2.0f * q0q1 +
    _2q2q3 - ay) - 4.0f * q1 * (1 - 2.0f * q1q1 - 2.0f * q2q2 - az) +
    _2bz * q3 * (_2bx * (0.5f - q2q2 - q3q3) + _2bz * (q1q3 - q0q2)
    - mx) + (_2bx * q2 + _2bz * q0) * (_2bx * (q1q2 - q0q3) + _2bz *
    (q0q1 + q2q3) - my) + (_2bx * q3 - _4bz * q1) * (_2bx * (q0q2 +
    q1q3) + _2bz * (0.5f - q1q1 - q2q2) - mz);
111 s2 = -_2q0 * (2.0f * q1q3 - _2q0q2 - ax) + _2q3 * (2.0f * q0q1 +
    _2q2q3 - ay) - 4.0f * q2 * (1 - 2.0f * q1q1 - 2.0f * q2q2 - az) +
    (-_4bx * q2 - _2bz * q0) * (_2bx * (0.5f - q2q2 - q3q3) + _2bz *
    (q1q3 - q0q2) - mx) + (_2bx * q1 + _2bz * q3) * (_2bx * (q1q2 -
    q0q3) + _2bz * (q0q1 + q2q3) - my) + (_2bx * q0 - _4bz * q2) * (
    _2bx * (q0q2 + q1q3) + _2bz * (0.5f - q1q1 - q2q2) - mz);
112 s3 = _2q1 * (2.0f * q1q3 - _2q0q2 - ax) + _2q2 * (2.0f * q0q1 +
    _2q2q3 - ay) + (-_4bx * q3 + _2bz * q1) * (_2bx * (0.5f - q2q2 -
    q3q3) + _2bz * (q1q3 - q0q2) - mx) + (-_2bx * q0 + _2bz * q2) * (

```

```

        _2bx * (q1q2 - q0q3) + _2bz * (q0q1 + q2q3) - my) + _2bx * q1 * (
        _2bx * (q0q2 + q1q3) + _2bz * (0.5f - q1q1 - q2q2) - mz);
113 recipNorm = invSqrt(s0 * s0 + s1 * s1 + s2 * s2 + s3 * s3); //
        normalise step magnitude
114 s0 *= recipNorm;
115 s1 *= recipNorm;
116 s2 *= recipNorm;
117 s3 *= recipNorm;
118
119 // Apply feedback step
120 qDot1 -= beta * s0;
121 qDot2 -= beta * s1;
122 qDot3 -= beta * s2;
123 qDot4 -= beta * s3;
124 }
125
126 // Integrate rate of change of quaternion to yield quaternion
127 q0 += qDot1 * (1.0f / sampleFreq);
128 q1 += qDot2 * (1.0f / sampleFreq);
129 q2 += qDot3 * (1.0f / sampleFreq);
130 q3 += qDot4 * (1.0f / sampleFreq);
131
132 // Normalise quaternion
133 recipNorm = invSqrt(q0 * q0 + q1 * q1 + q2 * q2 + q3 * q3);
134 q0 *= recipNorm;
135 q1 *= recipNorm;
136 q2 *= recipNorm;
137 q3 *= recipNorm;
138 }
139
140 //
-----

141 // IMU algorithm update
142
143 void MadgwickAHRSupdateIMU(float gx, float gy, float gz, float ax, float ay,
    float az) {
144     float recipNorm;
145     float s0, s1, s2, s3;
146     float qDot1, qDot2, qDot3, qDot4;
147     float _2q0, _2q1, _2q2, _2q3, _4q0, _4q1, _4q2, _8q1, _8q2, q0q0, q1q1,
        q2q2, q3q3;
148
149     // Rate of change of quaternion from gyroscope
150     qDot1 = 0.5f * (-q1 * gx - q2 * gy - q3 * gz);
151     qDot2 = 0.5f * (q0 * gx + q2 * gz - q3 * gy);
152     qDot3 = 0.5f * (q0 * gy - q1 * gz + q3 * gx);
153     qDot4 = 0.5f * (q0 * gz + q1 * gy - q2 * gx);
154
155     // Compute feedback only if accelerometer measurement valid (avoids NaN
        in accelerometer normalisation)
156     if(!((ax == 0.0f) && (ay == 0.0f) && (az == 0.0f))) {
157
158         // Normalise accelerometer measurement
159         recipNorm = invSqrt(ax * ax + ay * ay + az * az);

```

```

160     ax *= recipNorm;
161     ay *= recipNorm;
162     az *= recipNorm;
163
164     // Auxiliary variables to avoid repeated arithmetic
165     _2q0 = 2.0f * q0;
166     _2q1 = 2.0f * q1;
167     _2q2 = 2.0f * q2;
168     _2q3 = 2.0f * q3;
169     _4q0 = 4.0f * q0;
170     _4q1 = 4.0f * q1;
171     _4q2 = 4.0f * q2;
172     _8q1 = 8.0f * q1;
173     _8q2 = 8.0f * q2;
174     q0q0 = q0 * q0;
175     q1q1 = q1 * q1;
176     q2q2 = q2 * q2;
177     q3q3 = q3 * q3;
178
179     // Gradient decent algorithm corrective step
180     s0 = _4q0 * q2q2 + _2q2 * ax + _4q0 * q1q1 - _2q1 * ay;
181     s1 = _4q1 * q3q3 - _2q3 * ax + 4.0f * q0q0 * q1 - _2q0 * ay - _4q1 +
182         _8q1 * q1q1 + _8q1 * q2q2 + _4q1 * az;
183     s2 = 4.0f * q0q0 * q2 + _2q0 * ax + _4q2 * q3q3 - _2q3 * ay - _4q2 +
184         _8q2 * q1q1 + _8q2 * q2q2 + _4q2 * az;
185     s3 = 4.0f * q1q1 * q3 - _2q1 * ax + 4.0f * q2q2 * q3 - _2q2 * ay;
186     recipNorm = invSqrt(s0 * s0 + s1 * s1 + s2 * s2 + s3 * s3); //
187         normalise step magnitude
188     s0 *= recipNorm;
189     s1 *= recipNorm;
190     s2 *= recipNorm;
191     s3 *= recipNorm;
192
193     // Apply feedback step
194     qDot1 -= beta * s0;
195     qDot2 -= beta * s1;
196     qDot3 -= beta * s2;
197     qDot4 -= beta * s3;
198 }
199
200 // Integrate rate of change of quaternion to yield quaternion
201 q0 += qDot1 * (1.0f / sampleFreq);
202 q1 += qDot2 * (1.0f / sampleFreq);
203 q2 += qDot3 * (1.0f / sampleFreq);
204 q3 += qDot4 * (1.0f / sampleFreq);
205
206 // Normalise quaternion
207 recipNorm = invSqrt(q0 * q0 + q1 * q1 + q2 * q2 + q3 * q3);
208 q0 *= recipNorm;
209 q1 *= recipNorm;
210 q2 *= recipNorm;
211 q3 *= recipNorm;
212 }
213 //

```

```
-----
212 // Fast inverse square-root
213 // See: http://en.wikipedia.org/wiki/Fast_inverse_square_root
214
215 float invSqrt(float x) {
216     float halfx = 0.5f * x;
217     float y = x;
218     long i = *(long*)&y;
219     i = 0x5f3759df - (i>>1);
220     y = *(float*)&i;
221     y = y * (1.5f - (halfx * y * y));
222     return y;
223 }
224
225 //
=====
226 // END OF CODE
227 //
=====
```

C. Planos

Los planos del diseño presentado deben estar aquí. Generados por NX/Solid-edge.